NAME:PRENA .M REG NO:192524056
COURSE NAME:JAVA PROGRAMMING COURSE CODE:CSA 0924

CO5-AT2

A nightly ETL job must update stock quantities for 200,000 products read from a supplier feed. Explain, referencing round-trip cost, why wrapping repeated executeUpdate() calls with addBatch() and a single executeBatch() dramatically reduces total run time compared to committing after every single update.

AIM:

To update stock quantities for a large number of products (200,000 records) read from a supplier feed using JDBC, and to demonstrate why batching the updates with addBatch() and a single executeBatch() call is dramatically faster than executing and committing every update individually, in terms of network round-trip cost.

PSEUDOCODE:

```
START
CONNECT to the database
DISABLE auto-commit on the connection
PREPARE an UPDATE statement with a placeholder
    for quantity and product_id

READ supplier feed of 200,000 product records
FOR each record in the feed
    SET quantity and product_id parameters
    ADD this statement to the batch (addBatch)
    IF batch size reaches a chunk limit (e.g. 1000)
        EXECUTE the batch (executeBatch)
        COMMIT the transaction
        CLEAR the batch
    END IF
END FOR
EXECUTE and COMMIT any remaining statements in batch
CLOSE the connection
STOP
```

JAVA CODE:

```
import java.sql.*;

class StockBatchUpdate {
    public static void main(String[] args) throws Exception {
        String url = "jdbc:mysql://localhost:3306/inventory";
        String user = "root", pass = "password";
        int batchSize = 1000;

        try (Connection conn = DriverManager.getConnection(url, user, pass)) {
            conn.setAutoCommit(false);

            String sql = "UPDATE products SET quantity = ? WHERE product_id = ?";
            PreparedStatement ps = conn.prepareStatement(sql);
            int count = 0;
```

```
            for (SupplierRecord rec : SupplierFeed.readAll()) { // 200,000 records
                ps.setInt(1, rec.quantity);
                ps.setInt(2, rec.productId);
                ps.addBatch();          // queue statement, no round trip yet
                count++;

                if (count % batchSize == 0) {
                    ps.executeBatch();  // ONE round trip for 1000 rows
                    conn.commit();
                    ps.clearBatch();
                }
            }
            ps.executeBatch();          // flush remaining rows
            conn.commit();

            System.out.println("Updated " + count + " products successfully.");
        }
    }
}
```

INPUT:

A supplier feed containing 200,000 product records, each with a product_id and an updated stock quantity.

OUTPUT:

```
Updated 200000 products successfully.
```

With per-row commit, this job takes several minutes because of 200,000 separate network round trips. With addBatch()/executeBatch() in chunks of 1000, the same job finishes in a few seconds, since only about 200 round trips are made to the database.

RESULT / EXPLANATION:

Every call to executeUpdate() followed by a commit sends the SQL statement to the database server over the network, waits for the server to parse, execute, and commit it, and then waits for the acknowledgement to come back before the JDBC driver can move to the next statement. This wait for the request to travel to the server and the response to travel back is the round-trip cost, and it is dominated by network and I/O latency rather than by the actual work the database does. For 200,000 products updated one at a time, the driver pays this round-trip cost 200,000 times, and even a modest 2-5 millisecond round trip adds up to several minutes of pure waiting, on top of the overhead of 200,000 separate transaction commits, each of which forces the database to flush its transaction log to disk.

addBatch() does not send anything to the database; it simply queues each parameterized statement in memory on the client side. executeBatch() then sends the whole queued group of statements to the database in a single network call, and the server executes them together before returning one combined acknowledgement. This collapses what would have been 200,000 round trips into a small number of round trips, one per batch chunk (for example about 200 round trips for chunks of 1000 rows), and committing once per chunk instead of once per row also drastically cuts down on log-flush overhead. Because round-trip latency, not raw statement execution time, is the dominant cost when updating hundreds of thousands of

small rows, reducing the number of round trips is what makes batching dramatically faster than committing after every single update.